PRÁCTICA CONTRASEÑAS CIBERSEGURIDAD

Rodrigo Hernández Sánchez

1. Parte 1: Creación de frase de paso con Diceware. Genera correctamente frase de paso con Diceware utilizando 6 palabras, documenta todo el proceso con capturas claras (tiradas de dados), muestra la frase original y la versión mejorada con símbolos/mayúsculas.



La frase mejorada será: Ávid0-Gu@u_Arrastrándos€/Contenedor!Conyug4l:ErroR

2. Prueba todas las contraseñas solicitadas en la web, incluye capturas claras de cada resultado, analiza las diferencias entre tipos de contraseñas, responde todas las preguntas de forma adecuada

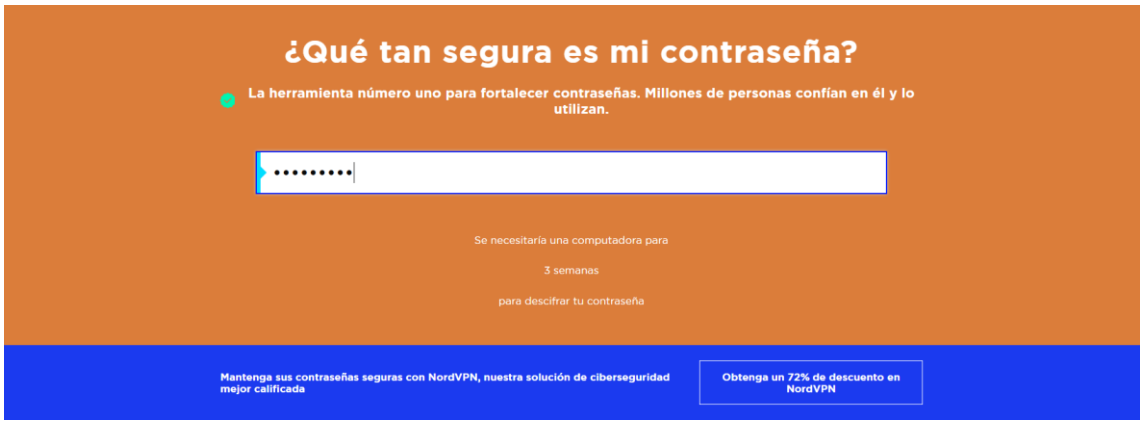
La web https://www.security.org/how-secure-is-my-password/ permite comprobar la fortaleza de diferentes tipos de contraseña.

Ve a la web, y compara cuanto tardaría en crackear diferentes contraseñas. Prueba, por ejemplo, "password123", "P@ssw0rd!", Tu frase de paso Diceware (sin modificaciones), Tu frase de paso Diceware con las mejoras, o cualquier otra que se te ocurra.

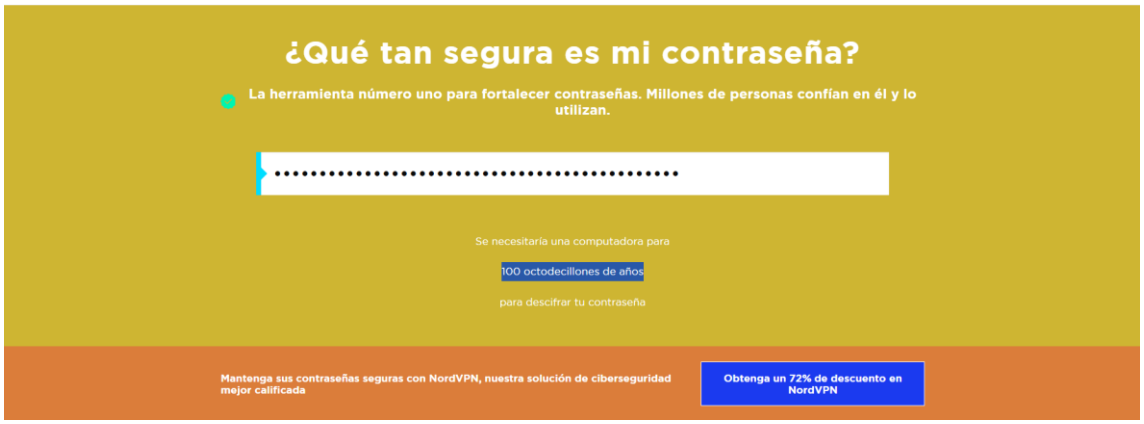
password123



P@ssw0rd!



AvidoGuauArrastrandoseContenedorConyugalError



Ávid0-Gu@u_Arrastrándos€/Contenedor!Conyug4l:ErroR



¿Cuánto tiempo tardaría un atacante en crackear cada una? ¿Qué diferencias notas entre una contraseña corta y compleja vs. una frase de paso larga? ¿Cuál es más fácil de recordar para ti?

password123 y P@ssw0rd!: se crackean al instante. Están ya en los diccionarios de contraseñas filtradas y las sustituciones típicas (a→@, o→0) ya están en las reglas de los crackers.

Corta y compleja frente a frase larga: la seguridad depende del “caos”, y la longitud aporta mucha más que la “rareza” de los caracteres. Una contraseña corta con símbolos va a ser predecible, mientras que una frase larga de palabras aleatorias es difícil de adivinar.

Es más fácil de recordar la de password123 o P@ssw0rd! debido a la combinación de caracteres que tiene y la habitualidad con la que usas/lees esas palabras.

3. Crea correctamente el usuario, examina /etc/passwd y /etc/shadow , explica detalladamente el significado de todos los campos principales, añade contraseña y verifica actualización, responde todas las preguntas de forma adecuada y documenta el almacenamiento en Windows con fuentes.

```
usuarioPrueba:x:1001:1001::/home/usuarioPrueba:/bin/sh
```

Usando el comando sudo useradd -m “Nombre” creamos el usuario.

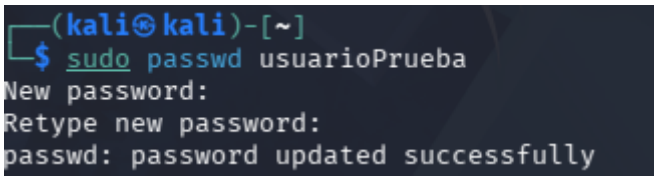
Campos principales:

1	**Nombre**	usuarioPrueba	Nombre de login.
2	**Contraseña**	x	El hash está en /etc/shadow.
3	**UID**	1001	ID numérico del usuario (usuario normal, el primero después de kali que es 1000).
4	**GID**	1001	ID del grupo primario, que useradd creó con el mismo número.
5	**GECOS**	*(vacío)*	Es un comentario, que, al no haberlo puesto, no aparece y se representa como un campo vacío.
6	**Home**	/home/usuarioPrueba	Directorio personal.
7	**Shell**	/bin/sh	Shell por defecto de useradd cuando no se indica -s.

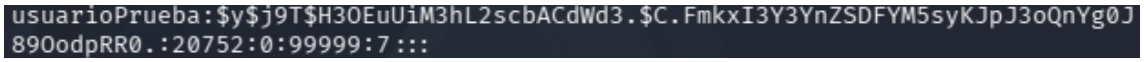
```
usuarioPrueba:!:20752:0:99999:7:::
```

Usando el comando sudo cat /etc/shadow creamos el usuario.

	Campo	Valor	Significado
1	Nombre	usuarioPrueba	Mismo nombre que en /etc/passwd
2	Hash	!	Cuenta bloqueada, sin contraseña. Nadie puede entrar con contraseña
3	Último cambio	20752	Días desde el 1/1/1970 hasta que se creó o cambió la contraseña. 20752 corresponde a hoy, 26/10/2026
4	Mínimo	0	Se puede cambiar la contraseña en cualquier momento
5	Máximo	99999	La contraseña nunca caduca (unos 273 años)
6	Aviso	7	Se avisa 7 días antes de que caduque
7	Inactividad	*(vacío)*	Sin días de gracia tras caducar
8	Expiración	*(vacío)*	La cuenta nunca se deshabilita
9	Reservado	*(vacío)*	Sin uso



Usamos el comando sudo psswd usuarioPrueba, nos pedirá una nueva contraseña y que la repitamos.



Volviendo a usar el comando sudo cat /etc/shadow vemos que se ha actualizado.

Dónde se guardan los hashes en Windows

Fichero: C:\Windows\System32\config\SAM

Formato: hash NT (MD4, sin sal), guardado en binario dentro del registro de Windows, no en texto como /etc/shadow.

En un dominio: el fichero es C:\Windows\NTDS\NTDS.dit.

En resumen, Windows guarda los hashes NT de las cuentas locales en la base de datos SAM (C:\Windows\System32\config\SAM), con formato MD4 (algoritmo de hash que se usa para ese cálculo) sin sal (valor aleatorio que se mezcla con la contraseña antes de calcular el hash).

4.1 Crea correctamente los 6 usuarios de prueba con contraseñas variadas, copia archivos `/etc/passwd` y `/etc/shadow` correctamente, combina con `unshadow` exitosamente, verifica el resultado con grep, documenta todo con capturas claras.

```
┌──(kali㉿kali)-[~]
└─$ mkdir practicaPasswords
```

```
┌──(kali㉿kali)-[~]
└─$ sudo cp /etc/passwd ~/practicaPasswords/passwd-test

┌──(kali㉿kali)-[~]
└─$ sudo cp /etc/shadow ~/practicaPasswords/shadow-test


┌──(kali㉿kali)-[~]
└─$ cd ~/practicaPasswords


┌──(kali㉿kali)-[~/practicaPasswords]
└─$ unshadow passwd-test shadow-test > passwords.txt

Created directory: /home/kali/.john
fopen: shadow-test: Permission denied
```

No funciona debido a que falta el sudo en el unshadow.

```
┌──(kali㉿kali)-[~/practicaPasswords]
└─$ sudo unshadow passwd-test shadow-test > passwords.txt

Created directory: /root/.john

┌──(kali㉿kali)-[~/practicaPasswords]
└─$ grep "testuser" passwords.txt
```

```
┌──(kali㉿kali)-[~/practicaPasswords]
└─$ john --format=crypt --wordlist=/usr/share/john/password.lst passwords.txt

Using default input encoding: UTF-8
Loaded 8 password hashes with 8 different salts (crypt, generic crypt(3) [?/6
4])
Cost 1 (algorithm [1:descrypt 2:md5crypt 3:sunmd5 4:bcrypt 5:sha256crypt 6:sh
a512crypt]) is 0 for all loaded hashes
Cost 2 (algorithm specific iterations) is 1 for all loaded hashes
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
password         (testUser1)
kali             (usuarioPrueba)
kali             (kali)
3g 0:00:02:29 DONE (2026-10-26 07:27) 0.02011g/s 23.77p/s 160.7c/s 160.7C/s !
@#$%..sss
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

4.2 Ejecuta correctamente ataque con diccionario básico, rockyou, aplica reglas, documenta resultados de cada ataque con capturas, crea tabla completa identificando qué contraseñas fueron crackeadas y cuáles no, analiza la efectividad de cada método

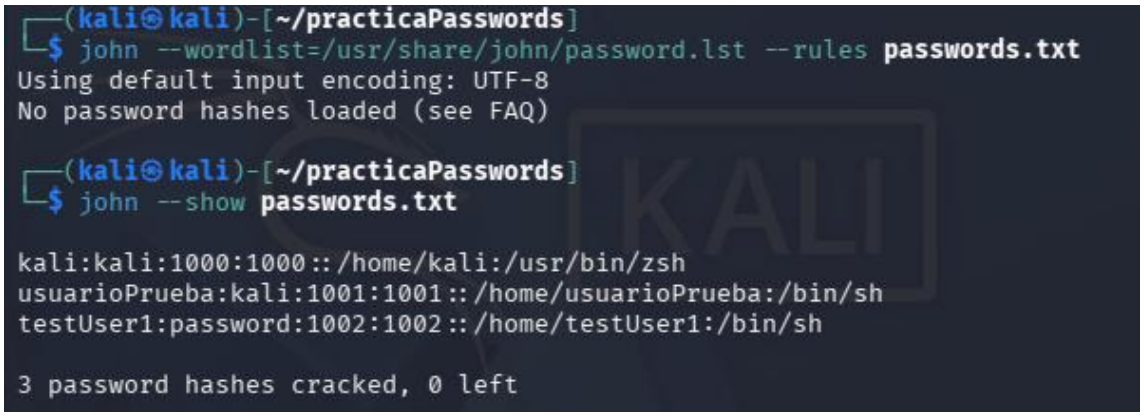
¿Qué contraseñas logró crackear John?

Logró crackear las contraseñas débiles debido a que aparecen en el diccionario con el que compara.

¿Qué contraseñas NO pudo crackear?

No logró comparar las contraseñas complejas debido a que usa un diccionario básico. Para crackear las complejas necesitaría más tiempo.

¿Logró crackear alguna contraseña adicional con las reglas?



No logró crackear ninguna contraseña más incluso con los parámetros nuevos introducidos.

Encuentra las tres que ya tenía guardadas debido a que compara el hash con lo guardado en el fichero caché: ~/.john/john.pot

5. Busca correctamente los 4 hashes MD5 proporcionados en CrackStation con capturas, genera hashes MD5/SHA1/SHA256 de al menos 4 contraseñas diferentes (admin, 123456, nombre propio, contraseña inventada), busca todos en CrackStation, responde todas las preguntas de forma adecuada.



Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, ripeMD160, whirlpool, MySQL 4.1+ (sha1(sha1_bin)), QubesV3.1BackupDefaults

hashes: 4 | cracked: 4 | partial: 0 | not found: 0 | invalid: 0

Hash	Type	Result
5f4dcc3b5aa765d61d8327deb882cf99	md5	password
098f6bcd4621d373cade4e832627b4f6	md5	test
e10adc3949ba59abbe56e057f20f883e	md5	123456
2b249a431be24e6ce35c117d97cd3130	md5	calasanz

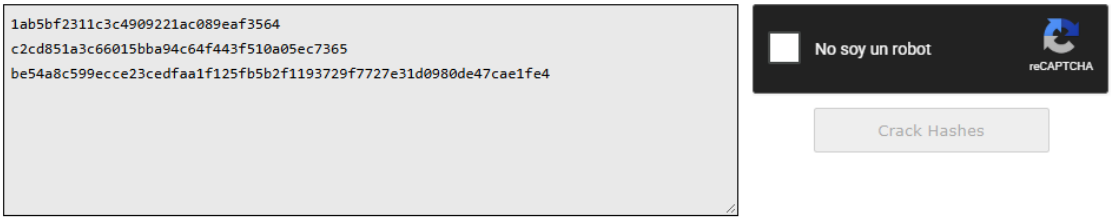
Color Codes: Green: Exact match, Yellow: Partial match — the hash column shows what that word really hashes to, with the part yours matched highlighted, Red: Not found.

¿Qué contraseñas encontró? ¿Fue instantáneo?

Password, test, 123456, Calasanz.

Sí, fue instantáneo.

¿Pudo CrackStation encontrar tus contraseñas personalizadas?



Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, ripeMD160, whirlpool, MySQL 4.1+ (sha1(sha1_bin)), QubesV3.1BackupDefaults

hashes: 3 | cracked: 3 | partial: 0 | not found: 0 | invalid: 0

Hash	Type	Result
1ab5bf2311c3c4909221ac089eaf3564	md5	mipassword
c2cd851a3c66015bba94c64f443f510a05ec7365	sha1	mipassword
be54a8c599ecce23cedfaa1f125fb5b2f1193729f7727e31d0980de47cae1fe4	sha256	mipassword

Color Codes: Green: Exact match, Yellow: Partial match — the hash column shows what that word really hashes to, with the part yours matched highlighted, Red: Not found.

Download CrackStation's Wordlist
1,493,677,782 words · 15 GB · torrent or HTTP

Command-Line Tool for Local Index Generation & Lookup
$ cargo install preimage

Sí, lo encontró en todos los resultados pasados.

¿Por qué las contraseñas comunes se encuentran inmediatamente?

Las contraseñas comunes se encuentran porque CrackStation no rompe el algoritmo matemáticamente ya que compara tu hash con una base de datos ya calculada de antemano, construida a partir de diccionarios y contraseñas reales filtradas en brechas de seguridad.

¿Qué limitación tienen las rainbow tables?

Solo encuentran lo que ya contienen ya que, si la contraseña es larga y aleatoria, sin aparecer nunca en ninguna filtración, no aparecerá.

La sal las inutiliza debido a que una rainbow table se calcula para un algoritmo sin sal, en donde si cada cuenta añade una sal aleatoria distinta, la misma contraseña produce un hash diferente en cada usuario, y habría que generar una tabla nueva para cada sal. Por eso el hash NT de Windows (sin sal) es vulnerable a este ataque y los hashes modernos de Linux (con sal) no.

Ocupan un espacio enorme ya que cubrir todas las combinaciones posibles de contraseñas de cierta longitud es inmanejable, así que las tablas solo cubren lo “probable”.

Los algoritmos lentos las frenan ya que, si el sistema usa un hash diseñado para ir despacio, construir o consultar una tabla se vuelve mucho más costoso que con MD5/SHA1, que son rápidos porque se diseñaron para otros usos, no para proteger contraseñas.

6. Propone política completa y detallada incluyendo: longitud mínima justificada, requisitos de complejidad específicos, período de expiración razonable, gestión de contraseñas anteriores, políticas de bloqueo tras intentos fallidos, considera usabilidad vs seguridad, justifica cada decisión basándose en lo visto en clase

1. Longitud mínima: 12 caracteres

En los ataques con John se vio que un diccionario encuentra en segundos cualquier palabra corta y común. Se aprecia que la longitud es el factor que más dificulta la fuerza bruta en donde cada carácter adicional multiplica el espacio de combinaciones posibles.

Ocho caracteres se considera el mínimo absoluto obsoleto hoy en día. Doce es el estándar recomendado actual para equilibrar seguridad y memorización.

2. Requisitos de complejidad: combinación de tipos, no reglas rígidas

Exigir al menos 3 de estos 4 grupos (mayúsculas, minúsculas, números, símbolos). No exigir un patrón fijo porque eso es predecible y reduce la variedad real.

3. Caducidad: 180 días (no menos), con aviso de 14 días

Una caducidad demasiado frecuente empuja a los usuarios a elegir contraseñas más débiles y predecibles (como por ejemplo, Octubre2026!, Noviembre2026!).

4. Gestión de contraseñas anteriores: recordar las últimas 5, con mínimo 1 día entre cambios

Evita la práctica de “cambiar la contraseña 2 veces seguidas para volver a la de siempre” cuando el sistema solo compara con la última. Recordar un histórico de 5 lo impide.

5. Bloqueo tras intentos fallidos: 5 intentos, bloqueo de 15 minutos

Esto es la defensa específica contra ataques de fuerza bruta online.

Cinco intentos es un número que tolera errores de escritura normales sin bloquear al usuario legítimo constantemente, pero frena un ataque automatizado de prueba y error.

***6.* autenticación en dos factores (2FA)**

Recomiendo complementar esta política con autenticación en dos factores (2FA) en los servicios que lo permitan. Es la medida que vimos que de verdad frena un ataque incluso si la contraseña se filtra, porque ni el diccionario, ni las reglas de John, ni CrackStation pueden saltarse un segundo factor.